Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: ” Python Web”

A efectuat : Midoni Lucian st. gr.SI – 264

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SCOPUL LUCRĂRII

Familiarizarea cu dezvoltarea aplicațiilor Python care interacționează cu servicii web reale, prin însușirea și aplicarea practică a conceptelor fundamentale de import al modulelor (din biblioteca standard și terțe), utilizare a metodelor și atributelor obiectelor, precum și definirea de funcții proprii modulare și reutilizabile.

MERSUL LUCRĂRII

Part1_imports

```python
# Exercitiul 1

import requests

print("Versiunea requests: ",requests.__version__)
import urllib.request



# Requests a avut nevoie deoarece nu face parte din bilioteca standard Python.
# Urllib nu a avut nevoie de pip install deoarece acesta este o biblioteca externa.
```

Figura 1.1/ Ex 1

```python
# Exercitiul 2

#      V1
import requests
response = requests.get(BASE_URL)
print("Cod de stare:",response.status_code)

#      V2
from requests import get
response_2= requests.get(BASE_URL)
print("Cod de stare:",response_2.status_code)

# Avantaj "import requests": Pastreaza codul clar si previne coliziuni de nume.

# Avantaj "from requests import get": Face codul mai scurt, mai curat și mai rapid de scris,
# si nu trebuie sa apelezi de mai multe ori functia.
```

Figura 1.2/ Ex 2

```python
# Exercitiul 3

import time
import requests as rq

time.sleep(1)

descarca3 = rq.get(BASE_URL, timeout=TIMEOUT)
print("Stare:",descarca3.status_code)


# Când un alias face codul mai usor de citit?
# Un alias face codul mai usor, atunci cand numele unei biblioteci e prea mare.

# Când un alias face codul mai greu de citit?
# Un alias face codul mai greu atunci cand folosesti un alias inventat pe care altii nu il inteleg
```

Figura 1.3/ Ex 3

```python
# Exercitiul 4


import time
import urllib.request

time.sleep(1)

descarca4 = urllib.request.urlopen(ECHO_URL, timeout=TIMEOUT)
print("Cod de stare (urllib):", descarca4.status)

corp_octeti = descarca4.read()
corp_caracter = corp_octeti.decode("utf-8")

print("Primele 200 de caractere: \n",corp_caracter[:200])


# Afisarea .staus
# Cod de stare (urllib): 200


#Afisarea primelor 200 de caractere

# Primele 200 de caractere:
#  <!DOCTYPE html>
# <html lang="en">
#
# <head>
#     <meta charset="UTF-8">
#     <title>httpbin.org</title>
#     <link href="https://fonts.googleapis.com/css?family=Open+Sans:400,700|Source+Code+Pro:300,600|Tit
```

Figura 1.4 / Ex 4

```python
# Exercitiul 5

import requests
print(dir(requests))

# get - functie , urllib3 - modul , Request - clasa
```

Figura 1.5/ Ex 5

```python
# Exercitiul 6

import time
import requests

help(requests.get)

time.sleep(1)

raspuns_6 = requests.get(BASE_URL, timeout=TIMEOUT)
print("Stare cod:",raspuns_6.status_code)
```

Figura 1.6/ Ex 6

```python
# Exercitiul 7

import time
import requests

time.sleep(1)

inainte = time.perf_counter()
rapsuns_7 = requests.get(BASE_URL, timeout=TIMEOUT)
dupa = time.perf_counter()

timp_cronometrat = rapsuns_7.elapsed
print("Timpul cronometrat de catre cerere este:", timp_cronometrat)
```

Figura 1.7/ Ex 7

```python
# Exercitiul 8


try:
    import bs4
    print("Modulul bs4 importat cu succes")

except ImportError:
    print("Instalati modulul cu: pip install beautifulsoup4")


# Modulul este un simplu file care contine colectii de functii si variabile globale cu extensia .py
# Pachetele sunt registre de adrese care contin colectii de module
# O bilioteca e o colectie de pachete si moduli care te ajuta sa rezolvi o problema
```

Figura 1.8/ Ex 8

Part2_methods

```python
# Exercitiul 9

import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)

print(response.status_code)
print(response.ok)
print(response.url)
print(response.encoding)
```

Figura 2.1/ Ex 9

```python
# Exercitiul 10

response = requests.get(BASE_URL, timeout=TIMEOUT)
response.raise_for_status()

try:
    bad_response = requests.get(
        BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT
    )
    bad_response.raise_for_status()
except requests.HTTPError:
    print("Pagina solicitata nu a fost gasita sau a aparut o eroare HTTP.")
```

Figura 2.2/ Ex 10

```python
# Exercitiul 11
import requests

response = requests.get(BASE_URL, timeout=TIMEOUT)

for header, value in response.headers.items():
    print(f"{header}: {value}")
```

Figura 2.3/Ex 11

```python
# Exercitiul 12
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)

print(response.headers.get("Server", "lipseste"))
print(response.headers.get("Content-Type", "lipseste"))
print(response.headers.get("content-type", "lipseste"))
```

Figura 2.4/Ex 12

```python
# Exercitiul 13
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)

count = response.text.lower().count("cyber")
print(count)
```

Figura 2.5/Ex 13

```python
# Exercitiul 14

import time
import requests

time.sleep(1)

response = requests.get(BASE_URL, timeout=TIMEOUT)

html = response.text

start = html.find("<title>") + len("<title>")
end = html.find("</title>")

titlu = html[start:end].strip()

print("Titlul paginii este:", titlu)
print("Afisare stare cod:",response.status_code)
```

Figura 2.6/Ex 14

```python
# Exercitiul 15

import time
import requests

time.sleep(1)

raspuns_15 = requests.get(BASE_URL, timeout=TIMEOUT)

linii = raspuns_15.text.splitlines()
numar_linii = len(linii)

cea_mai_lunga_linie = max(linii, key=len)
lungime_maxima = len(cea_mai_lunga_linie)

print("Numar lini:", numar_linii)
print("Lungimea cele mai lungi linii:", lungime_maxima)

print("Afisarea stare cod:",raspuns_15.status_code)
```

Figura 2.7/Ex 15

```python
# Exercitiul 16


import time
import requests

time.sleep(1)

response = requests.get(BASE_URL, timeout=TIMEOUT)

ver_https = response.url.startswith("https://")


if ver_https:
    print("Conexiunea este securizata (HTTPS).")
else:
    print("Conexiunea nu este securizata (HTTP).")

print("Code de stare:",response.status_code)
```

Figura 2.8/Ex 16

```python
# Exercitiul 17

import time
import requests

time.sleep(1)

raspuns_17 = requests.get( url: "http://cybercor.org", timeout=TIMEOUT)

print("Istoricul redirectionarii:")

for ist in raspuns_17.history:
    print(f"Cod stare: {ist.status_code} URL: {ist.url}")

print("URL final:", raspuns_17.url)
print("Codul de stare al adresei finale:", raspuns_17.status_code)
```

Figura 2.9/Ex 17

```python
# Exercitiul 18

import time
import requests

time.sleep(1)
response = requests.get(BASE_URL, timeout=TIMEOUT)

time.sleep(1)
response_1 = requests.head(BASE_URL, timeout=TIMEOUT)

comp = len(response.content)
comp_1 = len(response_1.content)

print("Lungimea .get :",comp)
print("Lungimea .head :",comp_1)


# requests.get() - reda atat antetele cat si continutul propriu zis
# requests.head() - reda doar antetele, de aceea este intotdeauna 0
```

Figura 2.10/Ex 18

```python
# Exercitiul 19

import time
import requests

time.sleep(1)
response = requests.get(BASE_URL, timeout=TIMEOUT)
cookies = response.cookies

if cookies:
        for cookie in cookies:
            print(f"Cookie setat: {cookie.name}  Securizat: {cookie.secure}")
else:
    print("Nici un cookie setat")
```

Figura 2.11/Ex 19

```python
# Exercitiul 20

import time
import requests

obiect = requests.Session()
obiect.headers.update({'User-Agent': 'WebLab-<Midoni Lucian>'})
time.sleep(1)

response = obiect.get(ECHO_URL+ "/headers", timeout=TIMEOUT)
print("Raspuns:\n ",response.json())

# response.text este un atribut ce deja are o valoare stocata de aceea nu are nevoie de paranteze.
# response.json() este o metoda ce executa un proces de interpretare motiv pentru care trebuie apelata
```

Figura 2.12/Ex 20

Part3_functions

```python
# Exercitiul 21

import time
import requests

def fetch(url):
    """Trimite o cerere GET către url și returnează obiectul"""
    time.sleep(1)
    response = requests.get(url, timeout=TIMEOUT)
    return response

raspuns = fetch(BASE_URL)
print("Cod de stare: ",raspuns.status_code)
```

Figura 3.1/Ex 21

```python
# Exercitiul 22

import time
import requests


def get_status(url):
    """Returnează doar codul de stare HTTP pentru adresa url."""
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code

apeluri = ["/","/robots.txt","/sitemap.xml"]

for apel in apeluri:
    time.sleep(1)
    url_complet = BASE_URL + apel
    stare = get_status(url_complet)
    print(f"Apelare: {apel:<13} | Cod de stare: {stare}")
```

Figura 3.2/Ex 22

```python
# Exercitiul 23

import time
import requests

def fetch(url, timeout=10):
    time.sleep(1)
    response = requests.get(url, timeout=timeout)
    return response

rasp1 = fetch(BASE_URL)
print("Afisarea cu valoare implicita 10s:", rasp1.status_code)

rasp2 = fetch(BASE_URL, timeout=3)
print("Afisarea cu valoare implicita 3s:", rasp2.status_code)
```

Figura 3.3/Ex 23

```python
# Exercitiul 24

import time
import requests

def get_title(html):
    """Primeste un sir html și returneaza titlul paginii."""
    start = html.find("<title>") + len("<title>")
    end = html.find("</title>")
    titlu = html[start:end].strip()
    return titlu

time.sleep(1)
response = requests.get(BASE_URL, timeout=TIMEOUT)

titlu_pagina = get_title(response.text)
print("Titlul paginii este:", titlu_pagina)
```

Figura 3.4/Ex 24

```python
# Exercitiul 25

import requests

def get_title(html):
    """Docstring"""
    start = html.find("<title>") + len("<title>")
    end = html.find("</title>")
    titlu = html[start:end].strip()
    return titlu

help(get_title)
```

Figura 3.5/Ex 25

```python
# Exercitiul 26

import time
import requests

def get_status(url: str) -> int:
    time.sleep(1)
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code
try:
    print(get_status(123))
except:
    print("Error: Adnotarilede tip str > int  nu se opresc")

# La rularea programului adnotarile nu se opresc
```

Figura 3.6/Ex 26

```python
# Exercitiul 27

import time
import requests

def page_exists(url):
    time.sleep(1)
    try:
        response = requests.get(url, timeout=TIMEOUT)
        return response.status_code == 200

    except requests.RequestException:
        return False

url_invalid = "https://this-domain-does-not-exist.invalid"
print("Exista domeniul invalid?:", page_exists(url_invalid))
```

Figura 3.7/Ex 27

```python
# Exercitiul 28

import time
import requests

def check_paths(base, paths):
    """Verifică o listă de căi pentru un URL de bază și returnează un dicționar {path: status_code}."""
    rezultate = {}

    for path in paths:
        time.sleep(1)
        url_complet = base + path
        response = requests.get(url_complet, timeout=TIMEOUT)
        rezultate[path] = response.status_code

    return rezultate

lista = ["/", "/robots.txt", "/sitemap.xml"]
dicționar_rezultat = check_paths(BASE_URL, lista)

print("Rezultat dicționar:", dicționar_rezultat)
```

Figura 3.8/Ex 28

```python
# Exercitiul 29

import time
import requests

def get_header(url,name,default="lipseste"):
    """Returnează valoarea unui antet."""
    time.sleep(1)
    response = requests.get(url,timeout=TIMEOUT)
    return response.headers.get(name, default)

antet = get_header(BASE_URL, name="Server")
print("Antetului serverului este:", antet)
```

Figura 3.9/Ex 29

```python
# Exercitiul 30

import time
import requests

def security_headers(url: str) -> dict:
    """Verifică prezența celor 5 antete de securitate și returnează un dicționar"""
    lista = ["Strict-Transport-Security",
             "Content-Security-Policy",
             "X-Frame-Options",
             "X-Content-Type-Options",
             "Referrer-Policy"
             ]

    time.sleep(1)
    response = requests.get(url, timeout=TIMEOUT)
    rezultat = {}
    for antet in lista:
        rezultat[antet] = antet in response.headers

    return rezultat

statut_securitate = security_headers(BASE_URL)
print("Prezența antetelor de securitate:\n", statut_securitate)
```

Figura 3.10/Ex 30

```python
import time
import requests


def score_headers(results: dict) -> str:
    """Primește dicționarul de rezultate"""
    prezente = sum(results.values())
    total = len(results)
    return f"{prezente}/{total}"

scor = score_headers(security_headers(BASE_URL))
print("Scorul antetelor de securitate:", scor)
```

Figura 3.11/Ex 31

```python
# Exercitiul 32

import time
import requests

def fetch_robots(base):
    """Descarcă conținutul /robots.txt sau returnează None."""
    time.sleep(1)  # Pauza de rețea obligatorie
    try:
        url = base.rstrip("/") + "/robots.txt"
        response = requests.get(url, timeout=TIMEOUT)
        if response.status_code == 200:
            return response.text
        return None
    except requests.RequestException:
        return None

def disallowed_paths(robots_text):
    """Extrage căile interzise dintr-un text robots.txt."""
    if not robots_text:
        return []

    cai = []
    for line in robots_text.splitlines():
        line = line.strip()
        if line.startswith("Disallow:"):
            cale = line[9:].strip()
            if cale:
                cai.append(cale)
    return cai

# Testare:
text = fetch_robots(BASE_URL)
rezultat = disallowed_paths(text)
print("Căi interzise:", rezultat)
```

Figura 3.12/Ex 32

```python
# Exercitiul 33

import time
import requests

def response_times(*urls: str) -> dict:
    """Măsoară timpul de răspuns pentru oricâte URL-uri primite ca argumente variable."""
    rezultate = {}

    for url in urls:
        time.sleep(1)
        try:
            response = requests.get(url, timeout=TIMEOUT)

            timp_secunde = response.elapsed.total_seconds()
            rezultate[url] = timp_secunde
        except requests.RequestException:
            rezultate[url] = None

    return rezultate

rezultat_timp = response_times(BASE_URL, BASE_URL + "/robots.txt")
print("Timpi de răspuns:", rezultat_timp)
```

Figura 3.13/Ex 33

```python
# Exercitiul 34

import time
import requests


def log(message, **details):

    parti = [message]

    for key, value in details.items():
        parti.append(f"{key}={value}")

        print(" | ".join(parti))

log("verificat", url=BASE_URL, status=200)
log("eroare rețea", url="https://invalid.url", code=404, retry=False)
```

Figura 3.14/Ex 34

Part4_modules

```python
# Exercitiul 35

import urllib.parse

url = "https://cybercor.org/path?x=1#top"
parsed = urllib.parse.urlparse(url)

print("Scheme:", parsed.scheme)
print("Netloc:", parsed.netloc)
print("Path:", parsed.path)
print("Query:", parsed.query)
print("Fragment:", parsed.fragment)
```

Figura 4.1/Ex 35

```python
# Exercitiul 36

import urllib.parse

BaseURL = "https://cybercor.org"

legaturi_relative = ["/about", "contact.html", "../index.html"]

for legatura in legaturi_relative:
    full_url = urllib.parse.urljoin(BaseURL, legatura)
    print(full_url)
```

Figura 4.2/Ex 36

```python
# Exercitiul 37

import re
import requests

def extract_links(html: str) -> list:  1 usage

    toate_legaturile = re.findall( pattern: r'href="([^"]+)"', html)
    legaturi_unice = []

    for link in toate_legaturile:
        if link not in legaturi_unice:
            legaturi_unice.append(link)

    return legaturi_unice

response = requests.get(BASE_URL, timeout=TIMEOUT)
links = extract_links(response.text)

print(f"Am gasit {len(links)} legături unice:\n")
for link in links:
    print(link)
```

Figura 4.3/Ex 37

```python
# Exercitiul 38
from urllib.parse import urljoin, urlparse

def split_links(links: list, domain: str):  1 usage
    interne = []
    externe = []
    target_netloc = urlparse(domain).netloc

    for link in links:
        full_url = urljoin(domain, link)
        link_netloc = urlparse(full_url).netloc

        if link_netloc == target_netloc:
            interne.append(full_url)
        else:
            externe.append(full_url)

    return interne, externe
response = requests.get(BASE_URL, timeout=TIMEOUT)

legaturi_interne, legaturi_externe = split_links( links: "https://cybercor.org", BASE_URL)

print(f"Legaturi interne:{len(legaturi_interne)}")
for link in legaturi_interne:
    print(link)

print(f"Legaturi externe:{len(legaturi_externe)}")
for link in legaturi_externe:
    print(link)
```

Figura 4.4/Ex 38

```python
# Exercitiul 39

import time
import requests
from html.parser import HTMLParser


class ImageFinder(HTMLParser):  3 usages

    def __init__(self):
        super().__init__()

        self.images = []

    def handle_starttag(self, tag : str , attrs : list[tuple[str, str | None]] ):  1 usage
        # codul vostru aici
        if tag == "img":
            src = dict [str, str | None] (attrs).get("src")
            if src is not None:
                self.images.append(src)

            print(self.handle_starttag(self.images,BASE_URL))
```

Figura 4.5/Ex 39

```python
# #----------------------------------------
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [

    "/logo.png",
    "poza.jpg",
    "https://cdn.example.com/banner.webp",
], "Parserul nu a găsit exact imaginile așteptate"

print("Testul a trecut!")
```

Figura 4.6/Ex 39

```python
#-----------------------------------------------------
response = requests.get(BASE_URL, timeout=TIMEOUT)
time.sleep(1)

finder = ImageFinder()
finder.feed(response.text)

print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
# #_____________________________________
time.sleep(1)
response = requests.get(BASE_URL, timeout=TIMEOUT)

real_finder = ImageFinder()
real_finder.feed(response.text)

numar_parser = len(real_finder.images)
numar_brut = response.text.lower().count("<img")

if numar_parser == numar_brut:
    print("Numerele coincid")
else:
    print("Numerele diferă")
```

Figura 4.7/Ex 39

```python
# Exercitiul 40

import hashlib
import requests
import time


def page_fingerprint(url):  2 usages
    time.sleep(1)
    response = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(response.content).hexdigest()

url_test = "https://cybercor.org"

print("Prima rulare: ", page_fingerprint(url_test))
time.sleep(1)
print("A doua rulare:", page_fingerprint(url_test))

# Rezultatele ar fi diferite atunci cand continutul paginii sar schimba
```

Figura 4.8/Ex 40

```python
#Exercitiul 41

import time
import json
import requests

def test_json_headers():  1 usage
    time.sleep(1)
    response = requests.get(BASE_URL, timeout=TIMEOUT)
    file = open("headers.json", "w")
    json.dump(dict [str, str] (response.headers), file, indent=2)
    file.close()

    file_citire = open("headers.json", "r")
    antete = antete = json.load(file_citire)
    file_citire.close()

    print(antete["Date"])

test_json_headers()
```

Figura 4.9/Ex 41

```python
# Exercitiul 42

import socket

def resolve(hostname):  1 usage
    return socket.gethostbyname(hostname)

url_test = "cybercor.org"

print("IP adress a Serverului:", url_test)
print(resolve(url_test))
```

Figura 4.10/Ex 42

```python
# Exercitiul 43

import socket
import ssl
from datetime import datetime

def cert_days_left(hostname):  1 usage

    context = ssl.create_default_context()
    sock = socket.create_connection((hostname, 443))
    wrap = context.wrap_socket(sock, server_hostname=hostname)

    data_text = wrap.getpeercert()["notAfter"]
    expirare_secunde = ssl.cert_time_to_seconds(data_text)
    expirare = datetime.fromtimestamp(expirare_secunde)
    timp_ramas = expirare - datetime.now()

    return timp_ramas

url_test = "cybercor.org"
zile  = cert_days_left(url_test)
print(f"Certificatul pentru {url_test} mai este valabil {zile} zile.")
```

Figura4.11/Ex 43

Partea 5

```python
# Exercitiul 44

import webtools
webtools.get_title(webtools.fetch(BASE_URL).text)
```

Figura 5.1/Ex 44

```python
# Exercitiul  45

import webtools
from webtools import get_title, security_headers

# Raspuns:
# In caz ca main.py va avea  functie proprie get_title()
# aceasta isi va da overwrite la afisare in terminal.
```

Figura 5.2/Ex 45

```python
# Exercitiul 46

if __name__ == "__main__":

    print("Autotest:", get_status("https://cybercor.org"))

# Atunci cand executam python webtools.py se seteaza automat variabila __name__ la valoarea __main__
# Dar atunci cand executam fisierul main.py fisierul webtools.py este importat, prin urmare
# variabila __name__ devine webtools iar conditia devine falsa si codul este ignorat.
```

Figura 5.3/Ex 46

```python
# Exercitul 47

from webtools import fetch, get_title, DEFAULT_HEADERS
print("Constanta DEFAULT_HEADERS este:", DEFAULT_HEADERS)
```

Figura 5.4/Ex 47

```python
# Exercitiul 48

import argparse

parser = argparse.ArgumentParser()
parser.add_argument( *name_or_flags: "url", help="https://cybercor.org")
args = parser.parse_args()

print(f"Url-ul este: {args.url}")
```

Figura 5.5/Ex 48

```python
# Exercitiul 49

import csv
from datetime import datetime
from webtools import check_paths

rezultate = check_paths(base=BASE_URL, paths=["/", "/robots.txt", "/sitemap.xml"])

with open("report.csv", mode="w", newline="") as fisier:
    writer = csv.writer(fisier)
    writer.writerow(["path", "status", "checked_at"])

    for path, status in rezultate.items():

        checked_at = datetime.now().isoformat()
        writer.writerow([path, status, checked_at])
print(rezultate)
```

Figura 5.6/Ex49

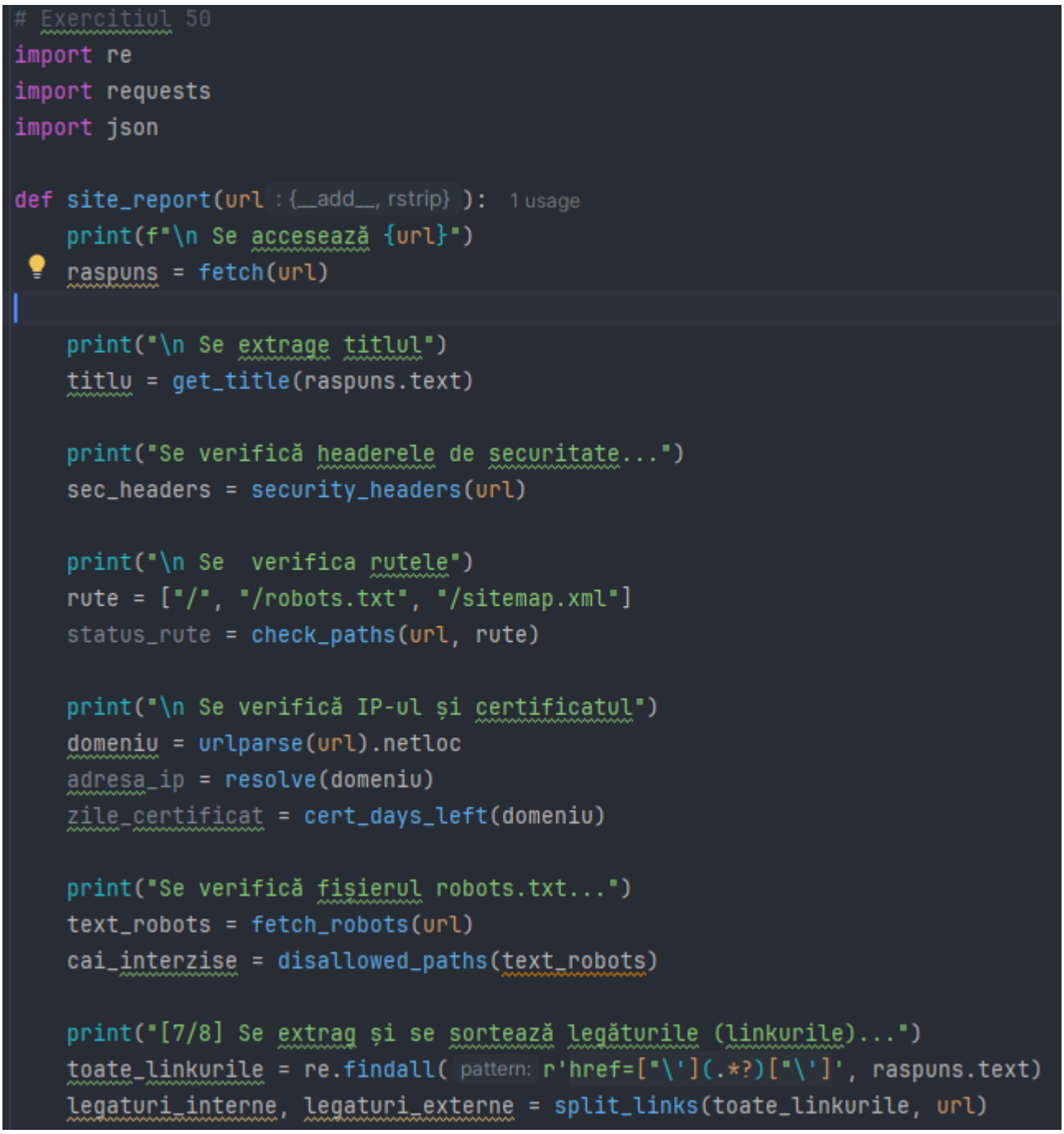
```python
# Exercitiul 50

import argparse
from webtools import site_report

parser = argparse.ArgumentParser()
parser.add_argument( *name_or_flags: "url",help="https://cybercor.org")
args = parser.parse_args()

site_report(args.url)
```

Figura 5.7/Ex 50 (main.py)



```python
# Exercitiul 50
import re
import requests
import json

def site_report(url :{__add__, rstrip} ):  1 usage
    print(f"\n Se accesează {url}")
    raspuns = fetch(url)

    print("\n Se extrage titlul")
    titlu = get_title(raspuns.text)

    print("Se verifică headerele de securitate...")
    sec_headers = security_headers(url)

    print("\n Se  verifica rutele")
    rute = ["/", "/robots.txt", "/sitemap.xml"]
    status_rute = check_paths(url, rute)

    print("\n Se verifică IP-ul și certificatul")
    domeniu = urlparse(url).netloc
    adresa_ip = resolve(domeniu)
    zile_certificat = cert_days_left(domeniu)

    print("Se verifică fișierul robots.txt...")
    text_robots = fetch_robots(url)
    cai_interzise = disallowed_paths(text_robots)

    print("[7/8] Se extrag și se sortează legăturile (linkurile)...")
    toate_linkurile = re.findall( pattern: r'href=["\'](.*?)["\']', raspuns.text)
    legaturi_interne, legaturi_externe = split_links(toate_linkurile, url)
```

Figura5.8/Ex 50

```python
print("Generarea Raportului")

raport = {
    "Site_url": url,
    "Cod de stare": raspuns.status_code,
    "Titlul paginii": titlu,
    "Adressa IP": url_test,
    "Rederictionari":[ist.status_code for ist in raspuns.history],
    "security_headers": sec_headers,
    "Certificat": zile,
    "Legaturi": [legaturi_interne,legaturi_externe],
    "Cai interzise": cai_interzise,
    "Salvat in report.json": True
}

print(" === RAPORT FINAL === ")
print(json.dumps(raport,indent=4))

with open("report.json", "w") as fisier_json:
    json.dump(raport, fisier_json, indent=4)

print("\n Raport creat cu succes")
```

Figura5.9/Ex 50

CONCLUZII

În cadrul acestui proiect de laborator, am aprofundat conceptele fundamentale ale limbajului Python aplicate în mediul web, învățând să gestionez eficient importurile, să apelez metode și să scriu funcții modulare și reutilizabile. Prin implementarea modulului propriu webtools.py și realizarea scriptului pentru generarea rapoartelor automate ale site-urilor web, mi-am consolidat abilitățile practice de trimitere a cererilor HTTP, analiză a datelor și tratare a erorilor, respectând totodată regulile esențiale de Securitate.

BIBLIOGRAFIE

1. Gemini – pentru structura si coerenta.

2. W3schools – pentru exemple: https://www.w3schools.com/python/default.asp